CAREFLOW CLINICAL AI • TECHNICAL SPECIFICATION

Reciprocal Rank Fusion (RRF)

Nguyên lý toán học, trực giác thiết kế và ứng dụng trong Hybrid Search RAG

1. Giới Thiệu & Bối Cảnh Thực Tế

Reciprocal Rank Fusion (RRF) là một thuật toán kết hợp thứ hạng (Rank Fusion) không giám sát (unsupervised), được phát minh bởi nhóm tác giả *Gordon V. Cormack, Charles L. A. Clarke và Stefan Büttcher*, công bố tại hội nghị danh giá hàng đầu về truy xuất thông tin **ACM SIGIR 2009**.

Trong kỷ nguyên của Trí tuệ nhân tạo tạo sinh và các hệ thống **RAG (Retrieval-Augmented Generation)**, RRF đã trở thành xương sống của mô hình **Hybrid Search**, chịu trách nhiệm dung hợp kết quả giữa hai phương pháp truy xuất độc lập:

- **Dense Vector Search (Ngữ nghĩa - Semantic):** Sử dụng Vector Embedding (OpenAI, Gemini Embedding, BGE) kết hợp cơ chế láng giềng gần nhất (HNSW, Cosine Similarity/Distance) để nắm bắt ý định người dùng ngay cả khi không trùng lặp câu chữ.
- **Sparse Lexical Search (Từ khóa - Lexical):** Sử dụng BM25 hoặc TF-IDF (PostgreSQL GIN tsvector, Elasticsearch) để truy xuất chính xác các thực thể riêng biệt: thuật ngữ y khoa, mã định danh ICD-10, tên hoạt chất thuốc, số hiệu xét nghiệm.

Vấn đề cốt lõi (Score Normalization Bottleneck): Làm sao để cộng điểm giữa một tài liệu có `Cosine Distance = 0.28` và một tài liệu có `BM25 Score = 14.82`? Hai thang đo điểm số hoàn toàn khác nhau về miền giá trị và phân phối thống kê. Chuẩn hóa Min-Max thường thất bại vì rất nhạy cảm với các giá trị ngoại lai (outliers).

Giải pháp đột phá của RRF: Hoàn toàn loại bỏ giá trị điểm số gốc (scores) và chỉ dựa trên **thứ hạng tương đối (relative ranks)** của các tài liệu trong từng danh sách kết quả.

2. Công Thức Toán Học & Tham Số k

CÔNG THỨC TỔNG QUÁT RECIPROCAL RANK FUSION

$$RRF_Score(d \in D) = \sum_{m \in M} [\ 1\ /\ (\ k\ +\ r_m(d)\ )\ ]$$

Các thành phần trong công thức:

- **D:** Tập hợp toàn bộ các tài liệu (chunks) được thu thập từ tất cả các hệ thống tìm kiếm.
- **M:** Tập hợp các bộ truy vấn độc lập (ví dụ: $M = \{\text{Dense Vector}, \text{Sparse BM25}\}$).
- **$r_m(d)$:** Thứ hạng (rank: $1, 2, 3, \dots, N$) của tài liệu d trong danh sách kết quả của hệ thống m. Nếu tài liệu không lọt vào top ứng viên của hệ thống m, số hạng tương ứng bằng 0.
- **k:** Hằng số làm mượt (Smoothing constant). Giá trị chuẩn mực được kiểm chứng qua hàng loạt thử nghiệm của Cormack et al. là **k = 60**.

Tại sao hằng số $k = 60$ lại tối ưu?

Nếu không có hằng số k (nghĩa là $k = 0$), điểm số nghịch đảo sẽ giảm cực kỳ dốc:

- Vị trí 1: $1 / 1 = 1.0$
- Vị trí 2: $1 / 2 = 0.5$ (giảm đột ngột 50%)
- Vị trí 3: $1 / 3 = 0.33$

Sự sụt giảm quá mạnh này sẽ khiến một tài liệu tình cờ đứng hạng 1 ở một hệ thống duy nhất sẽ "đè bẹp" một tài liệu có chất lượng cao và đứng hạng 2 ở cả hai hệ thống.

Khi đặt **k = 60**, độ dốc được làm dịu đáng kể:
• Vị trí 1: $1 / (60 + 1) = 1 / 61 \approx 0.01639$
• Vị trí 2: $1 / (60 + 2) = 1 / 62 \approx 0.01613$
• Vị trí 3: $1 / (60 + 3) = 1 / 63 \approx 0.01587$

Nhờ đó, **nguyên tắc vàng của RRF** được bảo toàn: *Tài liệu được nhiều hệ thống đồng thuận đánh giá cao sẽ luôn có điểm tổng hợp vượt trội hơn tài liệu chỉ xuất hiện đơn lẻ tại một hệ thống duy nhất.*

3. Ví Dụ Minh Họa Chi Tiết Từng Bước

Giả sử một bác sĩ lâm sàng thực hiện tra cứu: `"Phác đồ điều trị Đái tháo đường thai kỳ và nguy cơ tăng huyết áp"`. Hai hệ thống Dense Vector và Sparse BM25 độc lập trả về danh sách ứng viên Top 3 như sau:

Tài Liệu	Vector Search (Rank)	BM25 Search (Rank)	Tính Toán Điểm RRF (k=60)	Điểm Tổng
Doc B (Phác đồ BYT ĐTĐ Thai Kỳ)	**Hạng 2 (r = 2)**	**Hạng 1 (r = 1)**	**1/(60+2) + 1/(60+1)**	**0.03252 (Hạng 1)** 🥇
Doc A (Tổng quan ĐTĐ Type 2)	Hạng 1 (r = 1)	Hạng 3 (r = 3)	1/(60+1) + 1/(60+3)	**0.03226** (Hạng 2) 🥈
Doc D (Theo dõi huyết áp định kỳ)	Không xuất hiện	Hạng 2 (r = 2)	0 + 1/(60+2)	**0.01613** (Hạng 3) 🥉
Doc C (Chỉ số đường huyết lúc đói)	Hạng 3 (r = 3)	Không xuất hiện	1/(60+3) + 0	**0.01587** (Hạng 4)

Quan sát kết quả: Tài liệu **Doc B** không hề đứng đầu trong Vector Search (chỉ đứng hạng 2), nhưng nhờ xuất hiện trong nhóm đầu của **cả hai danh sách**, Doc B đã vươn lên giành vị trí quán quân (0.03252 điểm). Đây chính là ưu thế hội tụ tự nhiên của RRF.

4. Triển Khai Trong Mã Nguồn Thực Tế (CareFlow AI Service)

Dưới đây là phương thức triển khai RRF chuẩn mực trong dịch vụ `ClinicalHybridSearchService.java` của hệ sinh thái bệnh viện CareFlow:

```
@Service
@RequiredArgsConstructor
public class ClinicalHybridSearchService {

    public static final int RRF_K = 60;
    public static final int CANDIDATES_PER_SOURCE = 20;
    public static final int FINAL_TOP_K = 5;

    public List<RankedChunk> search(String queryText, String specialty) {
        Map<UUID, Double> rrfScores = new HashMap<>();
        Map<UUID, ChunkSearchResultProjection> chunkCache = new HashMap<>();

        // 1. DENSE VECTOR SEARCH (HNSW Cosine Distance)
        String embeddingString = geminiClientService.generateEmbedding(queryText);
        if (embeddingString != null) {
            List<ChunkSearchResultProjection> vectorResults =
                    chunkRepository.searchByVector(embeddingString, specialty, CANDIDATES_PER_SOURCE
            for (int rank = 1; rank <= vectorResults.size(); rank++) {
                ChunkSearchResultProjection chunk = vectorResults.get(rank - 1);
                double score = 1.0 / (RRF_K + rank);
                rrfScores.merge(chunk.getId(), score, Double::sum);
                chunkCache.putIfAbsent(chunk.getId(), chunk);
            }
        }

        // 2. SPARSE BM25 SEARCH (PostgreSQL GIN tsvector)
        List<ChunkSearchResultProjection> bm25Results =
                chunkRepository.searchByBm25(queryText.trim(), specialty, CANDIDATES_PER_SOURCE);
        for (int rank = 1; rank <= bm25Results.size(); rank++) {
            ChunkSearchResultProjection chunk = bm25Results.get(rank - 1);
            double score = 1.0 / (RRF_K + rank);
            rrfScores.merge(chunk.getId(), score, Double::sum);
            chunkCache.putIfAbsent(chunk.getId(), chunk);
        }

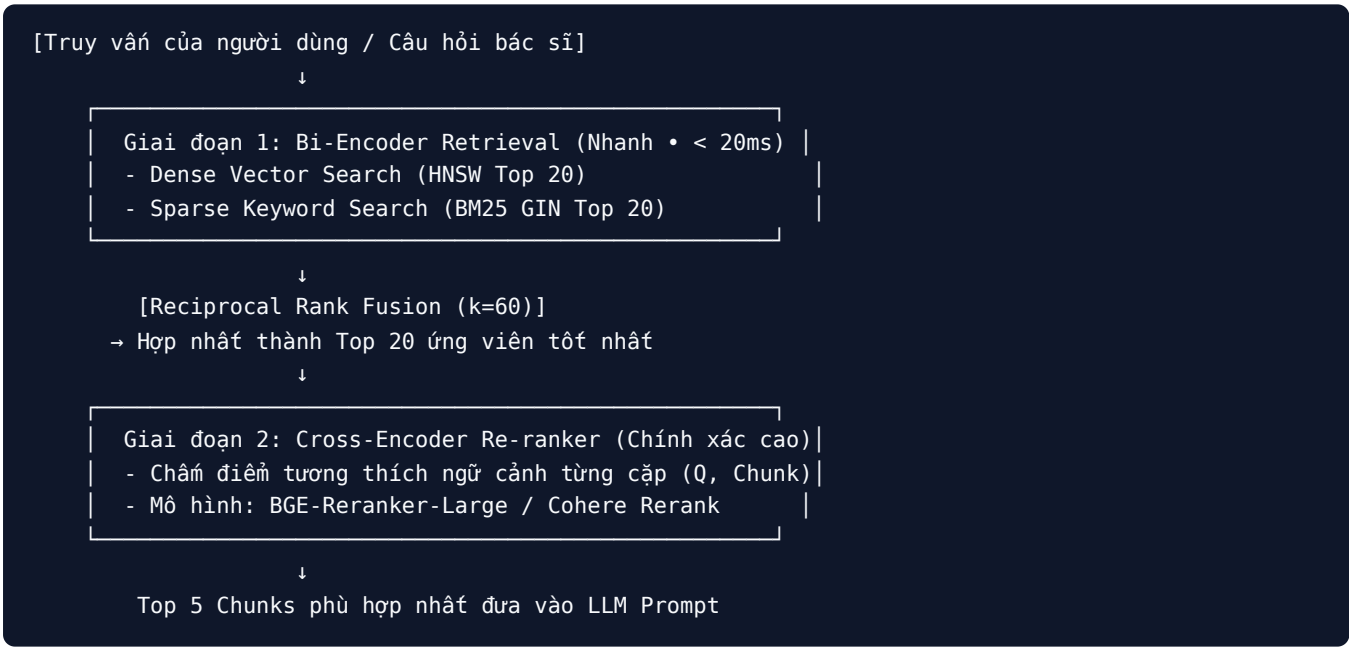
        // 3. SẮP XẾP VÀ TRÍCH XUẤT TOP K ỨNG VIÊN
        return rrfScores.entrySet().stream()
                .sorted(Map.Entry.<UUID, Double>comparingByValue().reversed())
                .limit(FINAL_TOP_K)
                .map(entry -> toRankedChunk(chunkCache.get(entry.getKey()), entry.getValue()))
                .toList();
    }
}
```

5. So Sánh & Kiến Trúc RAG Production Hoàn Chỉnh

Tiêu Chí	Weighted Score Fusion (Cộng điểm trọng số)	Reciprocal Rank Fusion (RRF)
Độ phức tạp cấu hình	Cao (cần tinh chỉnh α, β bằng tay)	**Không cần cấu hình (Parameter-free)**
Phụ thuộc thang đo	Rất nhạy cảm với scale điểm và phân phối	**Miễn nhiễm hoàn toàn với thang đo điểm**
Khả năng chịu lỗi (Resilience)	Dễ crash hoặc lệch điểm nếu 1 retriever trả về 0	**Tự động fallback mượt mà về retriever còn lại**
Độ tự tin (Confidence)	Giữ lại được độ lớn tuyệt đối của điểm số	Mất thông tin về khoảng cách thực tế giữa các rank

Kiến trúc Pipeline Hai Giai Đoạn (Two-stage Retrieval)

Để khắc phục nhược điểm "không đo lường độ tự tin tuyệt đối" của RRF, các hệ thống công nghiệp hiện đại kết hợp RRF cùng mô hình **Cross-Encoder Re-ranker** theo quy trình sau:



6. Kết Luận

Reciprocal Rank Fusion (RRF) là một giải pháp thanh lịch, trực quan và có cơ sở lý thuyết toán học vững chắc. RRF giải quyết triệt để vấn đề bất tương thích thang đo điểm số giữa các hệ thống truy xuất khác nhau mà không cần huấn luyện bổ sung. Đây là lựa chọn mặc định hàng đầu cho mọi kiến trúc Hybrid Search trong các hệ thống RAG hiện đại.

Tài liệu tham khảo: Cormack, G. V., Clarke, C. L., & Büttcher, S. (2009). *Reciprocal rank fusion outperforms Condorcet and individual rank learning methods*. In Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval (pp. 758-759).